PIMPRI CHINCHWAD EDUCATION TRUST's

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

(An Autonomous Institute)

DATABASE MANAGEMENT SYSTEM LABORATORY

NAME -	Kaustubh Nahire
PRN -	124B1F157
CLASS -	SY - IT
DIV -	A

ASSIGNMENT NO. 3 :- Joins, Sub-Queries and Views

Problem Statement: SQL Queries – All types of Join, Sub-Query and View: a. Write at least 10 SQL queries for suitable database application using SQL DML statements using joins and subqueries. b. Write queries using GROUP BY and HAVING clause.

Creating Table Users:

```
mysql> use ass3;
Database changed
mysql> CREATE TABLE Users (
    ->     user_id INT PRIMARY KEY,
    ->     name VARCHAR(50),
    ->     email VARCHAR(50),
    ->     phone VARCHAR(15),
    ->     password VARCHAR(50),
    ->     registration_date DATE
    -> );
Query OK, 0 rows affected (0.02 sec)
```

Creating Table Content:

```
mysql> CREATE TABLE Content (
    ->     content_id INT PRIMARY KEY,
    ->     title VARCHAR(100),
    ->     genre VARCHAR(50),
    ->     language VARCHAR(50),
    ->     duration INT,
    ->     release_date DATE,
    ->     age_rating VARCHAR(10)
    -> );
Query OK, 0 rows affected (0.02 sec)
```

Creating Table Subscription_Plan:

```
mysql> CREATE TABLE Subscription_Plan (
    ->     plan_id INT PRIMARY KEY,
    ->     plan_name VARCHAR(50),
    ->     price DECIMAL(10,2),
    ->     video_quality VARCHAR(20),
    ->     max_screens INT
    -> );
Query OK, 0 rows affected (0.02 sec)
```

Creating Table Subscription:

```
mysql> CREATE TABLE Subscription (
    ->     subscription_id INT PRIMARY KEY,
    ->     user_id INT,
    ->     plan_id INT,
    ->     start_date DATE,
    ->     end_date DATE,
    ->     status VARCHAR(20),
    ->     FOREIGN KEY (user_id) REFERENCES Users(user_id),
    ->     FOREIGN KEY (plan_id) REFERENCES Subscription_Plan(plan_id)
    -> );
Query OK, 0 rows affected (0.05 sec)
```

Creating Table Payment:

```
mysql> CREATE TABLE Payment (
    ->     payment_id INT PRIMARY KEY,
    ->     subscription_id INT,
    ->     payment_date DATE,
    ->     amount DECIMAL(10,2),
    ->     payment_method VARCHAR(20),
    ->     payment_status VARCHAR(20),
    ->     FOREIGN KEY (subscription_id) REFERENCES Subscription(subscription_id)
    -> );
Query OK, 0 rows affected (0.05 sec)
```

Creating Table Watch_History:

```
mysql> CREATE TABLE Watch_History (
    ->     history_id INT PRIMARY KEY,
    ->     user_id INT,
    ->     content_id INT,
    ->     watch_date DATE,
    ->     FOREIGN KEY (user_id) REFERENCES Users(user_id),
    ->     FOREIGN KEY (content_id) REFERENCES Content(content_id)
    -> );
Query OK, 0 rows affected (0.04 sec)
```

Inserting Values:

```
mysql> INSERT INTO Users VALUES
    -> (1, 'Amit', 'amit@gmail.com', '9876543210', '1234', '2024-01-01'),
    -> (2, 'Sneha', 'sneha@gmail.com', '9876543211', '1234', '2024-02-01'),
    -> (3, 'Rahul', 'rahul@gmail.com', '9876543212', '1234', '2024-03-01'),
    -> (4, 'Pooja', 'pooja@gmail.com', '9876543213', '1234', '2024-04-01'),
    -> (5, 'Karan', 'karan@gmail.com', '9876543214', '1234', '2024-05-01');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Content VALUES
    -> (101, 'Movie A', 'Action', 'English', 120, '2023-05-01', '18+'),
    -> (102, 'Movie B', 'Drama', 'Hindi', 140, '2023-06-10', '13+'),
    -> (103, 'Series C', 'Thriller', 'English', 45, '2023-07-15', '16+'),
    -> (104, 'Movie D', 'Comedy', 'Hindi', 110, '2023-08-20', 'U'),
    -> (105, 'Series E', 'Horror', 'English', 50, '2023-09-25', '18+');
Query OK, 5 rows affected (0.00 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Subscription_Plan VALUES
    -> (1, 'Basic', 199, 'HD', 1),
    -> (2, 'Standard', 299, 'Full HD', 2),
    -> (3, 'Premium', 499, '4K', 4),
    -> (4, 'Mobile', 149, 'SD', 1),
    -> (5, 'Family', 699, '4K', 6);
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

```
mysql> INSERT INTO Subscription VALUES
    -> (1, 1, 3, '2024-01-01', '2024-12-31', 'Active'),
    -> (2, 2, 1, '2024-02-01', '2024-08-01', 'Active'),
    -> (3, 3, 2, '2024-03-01', '2024-09-01', 'Expired'),
    -> (4, 4, 4, '2024-04-01', '2024-07-01', 'Active'),
    -> (5, 5, 5, '2024-05-01', '2025-05-01', 'Active');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Payment VALUES
    -> (1, 1, '2024-01-01', 499, 'UPI', 'Success'),
    -> (2, 2, '2024-02-01', 199, 'Card', 'Success'),
    -> (3, 3, '2024-03-01', 299, 'NetBanking', 'Failed'),
    -> (4, 4, '2024-04-01', 149, 'UPI', 'Success'),
    -> (5, 5, '2024-05-01', 699, 'Card', 'Success');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Watch_History VALUES
    -> (1, 1, 101, '2024-03-01'),
    -> (2, 2, 102, '2024-03-02'),
    -> (3, 3, 103, '2024-03-03'),
    -> (4, 4, 104, '2024-03-04'),
    -> (5, 5, 105, '2024-03-05');
Query OK, 5 rows affected (0.00 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Part a: SQL DML Queries using Joins and Subqueries

Query 1: JOIN —

```
mysql> SELECT u.name, p.plan_name
    -> FROM Users u
    -> JOIN Subscription s ON u.user_id = s.user_id
    -> JOIN Subscription_Plan p ON s.plan_id = p.plan_id;
+-------+-----------+
| name  | plan_name |
+-------+-----------+
| Amit  | Premium   |
| Sneha | Basic     |
| Rahul | Standard  |
| Pooja | Mobile    |
| Karan | Family    |
+-------+-----------+
5 rows in set (0.00 sec)
```

Query 2: LEFT JOIN —

LEFT JOIN returns all rows from the left table and matching rows from the right table. NULL appears where there is no match.

```
mysql> SELECT u.name, s.subscription_id
    -> FROM Users u
    -> LEFT JOIN Subscription s ON u.user_id = s.user_id;
+-------+----------------+
| name  | subscription_id |
+-------+----------------+
| Amit  |              1 |
| Sneha |              2 |
| Rahul |              3 |
| Pooja |              4 |
| Karan |              5 |
+-------+----------------+
5 rows in set (0.00 sec)
```

Query 3: RIGHT JOIN —

RIGHT JOIN returns all rows from the right table and matching rows from the left table.

```
mysql> SELECT u.name, s.subscription_id
    -> FROM Users u
    -> RIGHT JOIN Subscription s ON u.user_id = s.user_id;
+-------+----------------+
| name  | subscription_id |
+-------+----------------+
| Amit  |              1 |
| Sneha |              2 |
| Rahul |              3 |
| Pooja |              4 |
| Karan |              5 |
+-------+----------------+
5 rows in set (0.00 sec)
```

Query 4: FULL OUTER JOIN (using UNION) — Show all students and all halls whether matched or not

MySQL does not support FULL OUTER JOIN directly. It is achieved using LEFT JOIN UNION RIGHT JOIN.

```
mysql> SELECT u.name, s.subscription_id
    -> FROM Users u
    -> LEFT JOIN Subscription s ON u.user_id = s.user_id
    ->
    -> UNION
    ->
    -> SELECT u.name, s.subscription_id
    -> FROM Users u
    -> RIGHT JOIN Subscription s ON u.user_id = s.user_id;
+-------+-----------------+
| name  | subscription_id |
+-------+-----------------+
| Amit  |               1 |
| Sneha |               2 |
| Rahul |               3 |
| Pooja |               4 |
| Karan |               5 |
+-------+-----------------+
5 rows in set (0.01 sec)
```

Query 5: Query to Find the Payment with Highest Amount (Using Subquery)

```
mysql> SELECT *
    -> FROM Payment
    -> WHERE amount = (SELECT MAX(amount) FROM Payment);
+------------+-----------------+--------------+--------+----------------+----------------+
| payment_id | subscription_id | payment_date | amount | payment_method | payment_status |
+------------+-----------------+--------------+--------+----------------+----------------+
|          5 |               5 | 2024-05-01   | 699.00 | Card           | Success        |
+------------+-----------------+--------------+--------+----------------+----------------+
1 row in set (0.00 sec)
```

Query 6: INNER JOIN — Returns only the matching records from two or more tables based on a specified condition.

```
mysql> SELECT u.name, s.subscription_id
    -> FROM Users u
    -> INNER JOIN Subscription s ON u.user_id = s.user_id;
+-------+-----------------+
| name  | subscription_id |
+-------+-----------------+
| Amit  |               1 |
| Sneha |               2 |
| Rahul |               3 |
| Pooja |               4 |
| Karan |               5 |
+-------+-----------------+
5 rows in set (0.00 sec)
```

Query 7: Query to Display Users and the Content They Have Watched (Using JOIN)

```
mysql> SELECT u.name, c.title
    -> FROM Users u
    -> JOIN Watch_History w ON u.user_id = w.user_id
    -> JOIN Content c ON w.content_id = c.content_id;
+-------+----------+
| name  | title    |
+-------+----------+
| Amit  | Movie A  |
| Sneha | Movie B  |
| Rahul | Series C |
| Pooja | Movie D  |
| Karan | Series E |
+-------+----------+
5 rows in set (0.00 sec)
```

Query 8: Query to Find Users Who Have Watched Content (Using Subquery)

```
mysql> SELECT name
    -> FROM Users
    -> WHERE user_id IN (
    ->     SELECT user_id
    ->     FROM Subscription
    ->     WHERE plan_id = (
    ->         SELECT plan_id FROM Subscription_Plan WHERE plan_name = 'Premium'
    ->     )
    -> );
+------+
| name |
+------+
| Amit |
+------+
1 row in set (0.01 sec)
```

Query 9: Query to Display All Active Subscriptions

```
mysql> SELECT *
    -> FROM Subscription
    -> WHERE status = 'Active';
+-----------------+---------+---------+------------+------------+--------+
| subscription_id | user_id | plan_id | start_date | end_date   | status |
+-----------------+---------+---------+------------+------------+--------+
|               1 |       1 |       3 | 2024-01-01 | 2024-12-31 | Active |
|               2 |       2 |       1 | 2024-02-01 | 2024-08-01 | Active |
|               4 |       4 |       4 | 2024-04-01 | 2024-07-01 | Active |
|               5 |       5 |       5 | 2024-05-01 | 2025-05-01 | Active |
+-----------------+---------+---------+------------+------------+--------+
4 rows in set (0.00 sec)
```

Query 10: CREATE VIEW — Query to Find Users with Expired Subscriptions.

```
mysql> SELECT u.name, s.status
    -> FROM Users u
    -> JOIN Subscription s ON u.user_id = s.user_id
    -> WHERE s.status = 'Expired';
+-------+---------+
| name  | status  |
+-------+---------+
| Rahul | Expired |
+-------+---------+
1 row in set (0.01 sec)
```

Part b: Queries using GROUP BY and HAVING Clause

GROUP BY groups rows sharing the same values into summary rows. HAVING filters groups after aggregation (unlike WHERE which filters rows before grouping).

Query 1: GROUP BY — Total Payment per Subscription

```
mysql> SELECT subscription_id, SUM(amount) AS total_payment
    -> FROM Payment
    -> GROUP BY subscription_id;
+----------------+---------------+
| subscription_id | total_payment |
+----------------+---------------+
|              1 |        499.00 |
|              2 |        199.00 |
|              3 |        299.00 |
|              4 |        149.00 |
|              5 |        699.00 |
+----------------+---------------+
5 rows in set (0.00 sec)
```

Query 2: GROUP BY — Payment > 300

```
mysql> SELECT subscription_id, SUM(amount)
    -> FROM Payment
    -> GROUP BY subscription_id
    -> HAVING SUM(amount) > 300;
+----------------+-------------+
| subscription_id | SUM(amount) |
+----------------+-------------+
|              1 |      499.00 |
|              5 |      699.00 |
+----------------+-------------+
2 rows in set (0.00 sec)
```

Query 3: GROUP BY — Content Watch Count.

```
mysql> SELECT content_id, COUNT(*) AS views
    -> FROM Watch_History
    -> GROUP BY content_id;
+------------+-------+
| content_id | views |
+------------+-------+
|        101 |     1 |
|        102 |     1 |
|        103 |     1 |
|        104 |     1 |
|        105 |     1 |
+------------+-------+
5 rows in set (0.00 sec)
```

Query 4: GROUP BY with HAVING — Content watched more than once

HAVING is used to filter results after GROUP BY — it works on aggregated values.

```
mysql> SELECT content_id, COUNT(*)
    -> FROM Watch_History
    -> GROUP BY content_id
    -> HAVING COUNT(*) > 1;
Empty set (0.00 sec)
```

Query 5: GROUP BY with HAVING — Find exams that have seating for 3 or more students

```
mysql> CREATE VIEW User_Subscription_View AS
    -> SELECT u.name, p.plan_name, s.status
    -> FROM Users u
    -> JOIN Subscription s ON u.user_id = s.user_id
    -> JOIN Subscription_Plan p ON s.plan_id = p.plan_id;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM User_Subscription_View;
+-------+-----------+---------+
| name  | plan_name | status  |
+-------+-----------+---------+
| Amit  | Premium   | Active  |
| Sneha | Basic     | Active  |
| Rahul | Standard  | Expired |
| Pooja | Mobile    | Active  |
| Karan | Family    | Active  |
+-------+-----------+---------+
5 rows in set (0.00 sec)
```